

Travaux pratiques

Applications de la cryptographie

Construction et exploitation d'une infrastructure de gestion de clés (PKI)
avec OpenSSL et SoftHSM2

Outils OpenSSL 3.0.13, SoftHSM2 2.6.1, OpenSC (pkcs11-tool)
Système GNU/Linux (Ubuntu 24.04)
Trigramme APV
Date 22–23 juin 2026

Comment lire ce rapport. Les commandes réellement exécutées apparaissent sur fond sombre, les sorties OpenSSL correspondantes sur fond clair. Les encadrés **orange** signalent les points qui exigent un accès réseau « live » (Partie 1) ou une VM Windows Server (magasin de certificats, ADCS) : la *méthode* et les *commandes exactes* y sont fournies, seules les *valeurs* ou *captures* restent à compléter sur la plateforme cible.

Table des matières

1	Introduction et conventions	2
1.1	Objectifs	2
1.2	Environnement de travail	2
1.3	Choix de dimensionnement (recommandations ANSSI)	2
1.4	Durées de validité demandées	3
2	Partie 1 — Étude de certificats existants	4
2.1	Récupération du certificat (Q1)	4
2.2	Lecture de la chaîne (Q2 à Q5)	4
2.3	Analyse détaillée du certificat (Q6 à Q16)	4
2.4	La révocation (Q17 à Q21)	6
3	Partie 2 — Construction d'une PKI multi-autorités	8
3.1	Architecture cible	8
3.2	La certification croisée n'est que dans un sens (question du sujet)	8
3.3	Tâche 1 — Méthodologie de mise en place	8
3.3.1	Arborescence dédiée et fichiers de base de données	8
3.3.2	Fichier de configuration OpenSSL (un par AC)	9
3.3.3	Création de la racine RSA (auto-signée)	9
3.3.4	Création de la racine ECDSA (auto-signée)	10
3.3.5	Création des autorités subordonnées	10
3.4	Révocation par CRL (tâches 2 à 4)	11
3.5	Signature et chiffrement S/MIME (tâches 5 à 7)	11
3.6	Révocation par OCSP (tâches 8 à 12)	12
3.7	Certification croisée (tâches 13 et 14)	13
3.8	Magasin de certificats Windows (tâche 15)	14
4	Partie 3 — SoftHSM2 et ADCS	15
4.1	Autorité racine Linux avec SoftHSM2	15
4.2	CRL / ARL de la racine HSM	15
4.3	Absence d'export de clé privée	15
4.4	Point technique PKCS#11	15
4.5	Autorité subordonnée ADCS Windows	15
5	Conclusion	17
A	Annexe — Arborescence des fichiers produits	17
B	Annexe — Captures restantes à insérer	18

1 Introduction et conventions

1.1 Objectifs

Ce TP se décompose en trois parties complémentaires :

- **Partie 1 — Étude de certificats existants.** Récupération et analyse du certificat TLS du site <https://cyber.gouv.fr/>, lecture de la chaîne de certification, des extensions, de la CRL, et vérification de statut de révocation par CRL puis par OCSP.
- **Partie 2 — Construction d'une PKI multi-autorités.** Mise en place d'une hiérarchie inspirée de Let's Encrypt (deux racines, RSA et ECDSA, deux autorités subordonnées), avec révocation par CRL et par OCSP, échange S/MIME et *certification croisée* entre les deux racines.
- **Partie 3 — Racine adossée à un HSM.** Création d'une autorité racine dont la clé privée ne quitte jamais un module cryptographique (HSM virtuel SoftHSM2, via PKCS#11), puis signature d'une autorité subordonnée Microsoft ADCS.

1.2 Environnement de travail

```
$ openssl version
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
```

Le travail est placé sur la machine de rendu dans `/home/aurelien/Documents/certificats`.

Les paquets suivants ont été installés pour la Partie 3 : `softhsm2`, `opensc` (fournit `pkcs11-tool`) et `libengine-pkcs11-openssl` (moteur PKCS#11 pour OpenSSL).

1.3 Choix de dimensionnement (recommandations ANSSI)

Le sujet impose de choisir les algorithmes, tailles de clés et fonctions de hachage « en fonction des recommandations de l'ANSSI en vigueur ». Le document de référence est le *Guide des mécanismes cryptographiques* (RGS, annexe B1). Les règles mobilisées ici sont : module RSA d'au moins 2048 bits (3072 recommandé au-delà de 2030), clés sur courbes elliptiques d'au moins 256 bits, et fonction de hachage de la famille SHA-2 (au moins 256 bits). Nos choix :

Entité	Algorithme	Taille / courbe	Hachage
RSA_Root_CA_APV (racine)	RSA	4096 bits	SHA-256
Sub_RSA_CA_1_APV (sous-AC)	RSA	3072 bits	SHA-256
EC_Root_CA_APV (racine)	ECDSA	secp384r1 (P-384)	SHA-256
Sub_EC_CA_1_APV (sous-AC)	ECDSA	secp384r1 (P-384)	SHA-256
Entité finale RSA	RSA	3072 bits	SHA-256
Entité finale EC	ECDSA	prime256v1 (P-256)	SHA-256
Racine HSM (Partie 3)	RSA	4096 bits	SHA-256

Note

Nuances ANSSI à mentionner à l'oral. (1) L'ANSSI recommande de préférence les courbes *brainpool* (ex. `brainpoolP384r1`) ; nous avons retenu les courbes NIST (P-384/P-256) pour la compatibilité avec les magasins Windows et les navigateurs, ce qui correspond aussi au choix réel de Let's Encrypt. (2) Avec une clé P-384, le hachage « naturellement apparié » serait SHA-384 ; SHA-256 reste néanmoins conforme et a été retenu par homogénéité sur toute la PKI.

1.4 Durées de validité demandées

Objet	Durée	Valeur <code>-days</code> / CRL
AC racines	15 ans	5479 jours
AC subordonnées	10 ans	3653 jours
Entités finales	1 an	365 jours
CRL / ARL	1 mois	<code>default_crl_days</code> = 30
Racine HSM (Partie 3)	10 ans	3653 jours
AC subordonnée ADCS	5 ans	1825 jours

Note

Convention de nommage. Le DN commun à toutes les entités est `C=FR, O=TP IMT Mines Ales, OU=PKI APV`, le CN portant le nom de l'entité (ex. `RSA Root CA APV`). Le trigramme retenu pour ce compte rendu est APV.

Note

Simplification assumée sur les clés d'entité finale. Pour fluidifier le TP, les clés privées des entités finales sont générées *non chiffrées* (`encrypt_key = no`). En production, ces clés doivent être protégées par phrase de passe ou, mieux, conservées dans un module matériel — ce qui est précisément l'objet de la Partie 3.

2 Partie 1 — Étude de certificats existants

Les valeurs ci-dessous reprennent les résultats observés dans les fichiers de sortie du TP. Les captures navigateur restent à ajouter lorsqu'elles sont demandées explicitement par le sujet.

2.1 Récupération du certificat (Q1)

Q1. Quel protocole sécurise la connexion dans un navigateur ?

Réponse

La connexion HTTPS est sécurisée par **TLS**. La connexion OpenSSL observée a utilisé TLSv1.3. SSL est l'ancien nom historique ; aujourd'hui, le protocole utilisé est TLS.

```
openssl s_client -connect cyber.gouv.fr:443 -servername cyber.gouv.fr -showcerts \
2>/dev/null </dev/null > cyber.gouv.fr.chain.txt
awk '/BEGIN CERTIFICATE/{c++} END{print c}' cyber.gouv.fr.chain.txt
openssl s_client -connect cyber.gouv.fr:443 -servername cyber.gouv.fr 2>/dev/null </dev/null \
| openssl x509 -out cyber.gouv.fr.pem
```

2

2.2 Lecture de la chaîne (Q2 à Q5)

Réponse

Q2. Longueur de la chaîne : la chaîne fournie par `openssl s_client` contient **2 certificats** : le certificat serveur `cyber.gouv.fr` et le certificat subordonné **GandiCert**. La racine n'est pas envoyée par le serveur car elle est supposée présente dans le magasin de confiance du client.

Q3. AC racine : DigiCert Global Root G2. Elle n'est pas fournie dans la chaîne TLS récupérée, mais constitue l'ancrage de confiance de la chaîne **GandiCert**.

Q4. AC subordonnée : le certificat serveur est émis par `C=FR, O=Gandi SAS, CN=GandiCert`.

Q5. Taille de la clé publique du site : le certificat `cyber.gouv.fr` contient une clé publique **RSA 4096 bits**.

2.3 Analyse détaillée du certificat (Q6 à Q16)

```
openssl x509 -in cyber.gouv.fr.pem -noout -text > cyber.gouv.fr.x509.txt
```

Q6. Type de certificat.

Réponse

Il s'agit d'un certificat X.509 v3 d'entité finale pour serveur TLS. Le sujet contient `CN=cyber.gouv.fr`. Les SAN observés sont `DNS:cyber.gouv.fr`, `DNS:ssi.gouv.fr`, `DNS:www.ssi.gouv.fr` et `DNS:www.cyber.gouv.fr`. L'Extended Key Usage contient `TLS Web Server Authentication` et les Basic Constraints indiquent `CA:FALSE`.

Q7. Peut-il signer d'autres certificats ?

Réponse

Non. Le certificat contient Basic Constraints: CA:FALSE. Son Key Usage vaut Digital Signature, Key Encipherment et ne contient pas keyCertSign. Il ne peut donc pas signer d'autres certificats.

Q8. Peut-il signer du courrier électronique ?**Réponse**

Non pour l'usage S/MIME. L'Extended Key Usage contient uniquement TLS Web Server Authentication et ne contient pas E-mail Protection (emailProtection).

Q9. Où trouve-t-on la CRL de l'AC émettrice ?**Réponse**

Les points de distribution CRL réellement présents sont :

```
http://crl3.digicert.com/GandiCert.crl
http://crl4.digicert.com/GandiCert.crl
```

Q10. Format du numéro de série et conversion décimale.**Réponse**

Le numéro de série est affiché en hexadécimal par OpenSSL.

```
Serial hex avec separateurs : 05:7e:1f:dd:c4:8e:7a:07:80:64:3a:be:eb:cc:52:a5
Serial hex normalise       : 057E1FDDC48E7A0780643ABEEBCC52A5
Serial decimal             : 7301015708052902158374241928634716837
```

Il s'agit d'un entier ASN.1 affiché en hexadécimal ; la valeur décimale montre qu'il s'agit d'un grand nombre non trivial à prédire.

Q11. Afficher la clé publique.

```
openssl x509 -in cyber.gouv.fr.pem -pubkey -noout
```

```
Public Key Algorithm: rsaEncryption
Public-Key: (4096 bit)
Exponent: 65537 (0x10001)
```

Q12. Afficher la clé privée.**Réponse**

C'est impossible à partir du certificat public. Un certificat X.509 contient la clé publique, l'identité, les extensions et la signature de l'autorité, mais jamais la clé privée. Celle-ci reste uniquement côté serveur.

Q13 à Q16. Lecture et vérification de la CRL.

```
wget -O GandiCert.crl http://crl3.digicert.com/GandiCert.crl
openssl crl -inform DER -in GandiCert.crl -text -noout > crl.txt
rg "057E1FDDC48E7A0780643ABEEBCC52A5|05:7E:1F:DD:C4:8E:7A:07:80:64:3A:BE:EB:CC:52:A5" crl.txt
```

Réponse

Q13. La CRL consultée indique :

```
Last Update: Jun 22 12:45:56 2026 GMT
Next Update: Jun 29 12:45:56 2026 GMT
```

Sa durée de validité est donc de **7 jours**.

Q14. Les premières entrées affichées dans `crl.txt` ne montrent que le numéro de série et la date de révocation. Des champs `X509v3 CRL Reason Code` apparaissent plus loin, mais pas systématiquement pour les premières entrées.

Q15. Pour vérifier rapidement que la CRL est la bonne, on contrôle que son **Issuer** correspond à l'AC émettrice du certificat (`C=FR, O=Gandi SAS, CN=GandiCert`), que l'URL provient bien des `CRL Distribution Points`, puis que la signature de la CRL est vérifiable avec le certificat de cette AC.

Q16. La recherche du numéro de série du certificat `cyber.gouv.fr` dans la CRL ne renvoie aucun résultat : le certificat n'apparaît donc pas comme révoqué dans la CRL consultée.

2.4 La révocation (Q17 à Q21)

Site `https://revoked-rsa-dv.ssl.com/` dans 3 navigateurs.

SSL.com - Test Website

This is a test website authenticated by [SSL.com](https://ssl.com).

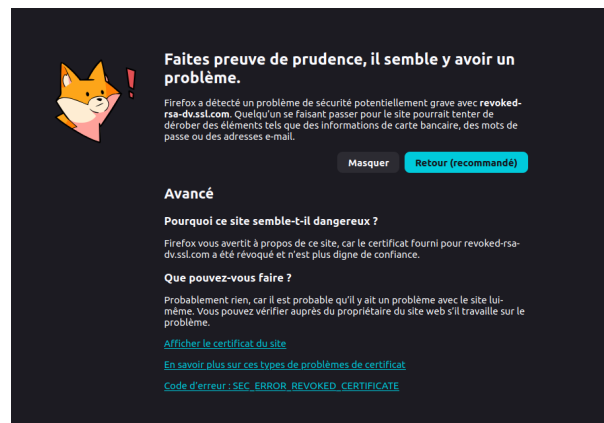


Figure 1: Pages d'erreur de sécurité pour `https://revoked-rsa-dv.ssl.com` : à gauche Chromium/Chrome (`NET::ERR_CERT_REVOKED`), à droite Firefox (`SEC_ERROR_REVOKED_CERTIFICATE`).

Q17. Le série du certificat révoqué est-il dans la CRL ?

Réponse

Oui. Le certificat testé est `01-public-certs/revoked-rsa-dv.ssl.com.pem`. Son numéro de série est :

```
1811B09C4BB9E179133BB9D9A9B140C5
```

La CRL utilisée est `http://crls.ssl.com/SSL.com-TLS-I-RSA-R1.crl`. Le numéro y apparaît avec la date de révocation suivante :

```
Serial Number: 1811B09C4BB9E179133BB9D9A9B140C5
Revocation Date: Jun 9 14:37:38 2026 GMT
```

Q18. Vérification OCSP.

```
openssl ocsf \
  -issuer ssl.com-tls-i-rsa-r1.pem \
  -cert revoked-rsa-dv.ssl.com.pem \
  -url http://ocsp.ssl.com \
  -no_nonce \
  -resp_text -text
```

```
OCSP Response Status: successful (0x0)
Cert Status: revoked
Revocation Time: Jun  9 14:37:38 2026 GMT
revoked-rsa-dv.ssl.com.pem: revoked
```

Réponse

La CRL et OCSP concordent : le certificat est bien révoqué.

Q19. Numéros de série de google.com et youtube.com.

```
google.com : 139CDF29A8B5BC5212413EEEAFFD18CBE
youtube.com : 139CDF29A8B5BC5212413EEEAFFD18CBE
```

Réponse

Les deux hôtes ont présenté le même certificat feuille, émis par C=US, O=Google Trust Services, CN=WR2, avec CN=*.google.com. Les SAN contiennent à la fois DNS:google.com et DNS:youtube.com. Il s'agit donc d'un certificat multi-SAN partagé par l'infrastructure Google.

Q20. https://wrong.host.badssl.com/.

```
verify error:num=62:hostname mismatch
Verification error: hostname mismatch
```

Réponse

L'alerte est cohérente : le nom demandé `wrong.host.badssl.com` ne correspond pas au certificat présenté. La validation du nom d'hôte repose sur les SAN, ou à défaut sur le CN selon la logique de la RFC 6125. Un wildcard ne couvre qu'un seul niveau de sous-domaine.

Q21. Certificats racines ISRG.

Réponse

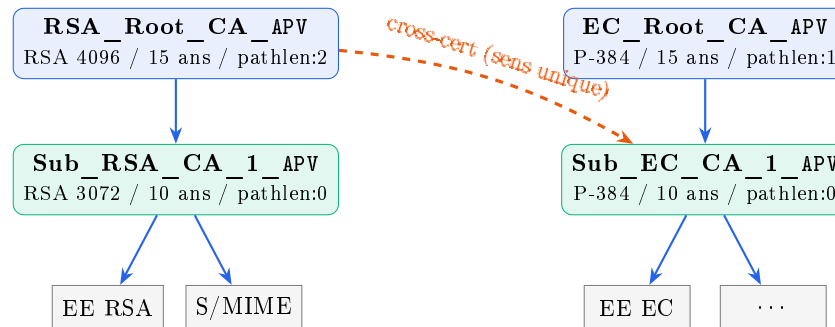
Non réalisé : aucun fichier `isrg-1.crt` ou `isrg-2.crt` n'a été trouvé dans l'arborescence de travail. La méthode de lecture reste :

```
for f in isrg-1.crt isrg-2.crt; do
  openssl x509 -in $f -noout -subject -issuer -dates
  openssl x509 -in $f -noout -text | grep -E "Public Key Algorithm|Public-Key|Signature"
  ↪ Algorithm"
done
```

3 Partie 2 — Construction d'une PKI multi-autorités

3.1 Architecture cible

La hiérarchie reproduit (en simplifié) celle de Let's Encrypt avant juin 2024 : deux racines indépendantes, l'une RSA, l'autre ECDSA, chacune signant une autorité subordonnée, les subordonnées signant les entités finales. Une *certification croisée unidirectionnelle* relie les deux branches.



Contraintes de longueur de chemin (pathLenConstraint). Le *pathlen* borne le nombre d'AC intermédiaires pouvant suivre dans le chemin (hors entité finale). On applique : racine RSA **pathlen:2** (deux niveaux de sous-AC autorisés), racine EC **pathlen:1** (un seul niveau), et les deux subordonnées **pathlen:0** (elles ne signent que des entités finales).

3.2 La certification croisée n'est que dans un sens (question du sujet)

Réponse

Ce qu'induit précisément un sens unique. Une certification croisée consiste à faire signer la clé publique d'une racine par l'autre racine, ce qui produit un certificat supplémentaire pour la même autorité. Ici, **RSA_Root_CA_APV** signe un certificat croisé pour **EC_Root_CA_APV**. Un vérificateur qui fait confiance à la racine RSA peut donc valider une chaîne de la branche EC via ce certificat croisé. L'inverse n'est pas vrai sans certificat miroir **RSA_Root_CA_APV** signé par **EC_Root_CA_APV**.

Réponse

Exemple de chaîne croisée unidirectionnelle :

```
RSA_Root_CA_APV
-> EC_Root_CA_APV (certificat croise signe par RSA_Root_CA_APV)
-> Sub_EC_CA_1_APV
-> Leaf_EC_1_APV.crosspath
```

Pour une certification croisée dans les deux sens, il faudrait émettre un second certificat croisé, cette fois avec la racine EC comme émetteur et la clé publique de la racine RSA comme sujet :

```
RSA_Root_CA_APV -> EC_Root_CA_APV
EC_Root_CA_APV  -> RSA_Root_CA_APV
```

3.3 Tâche 1 — Méthodologie de mise en place

3.3.1 Arborescence dédiée et fichiers de base de données

Chaque AC dispose de son répertoire, avec **private/** (clé privée, **chmod 700**) et **db/** (base de données OpenSSL). Conformément au sujet, **quatre fichiers doivent exister avant tout appel**

à `openssl ca` : la base (`.db`, vide), ses attributs (`.db.attr`, vide), le compteur de série (`.srl = 01`) et le compteur de CRL (`.crl.srl = 01`).

```
mkdir -p ca/{certs,crl,newcerts,csr,smime,ocsp}
for CA in RSA_Root_CA EC_Root_CA Sub_RSA_CA_1 Sub_EC_CA_1; do
  mkdir -p ca/$CA/db ca/$CA/private
  chmod 700 ca/$CA/private
  : > ca/$CA/db/$CA.db          # base vide mais existante
  : > ca/$CA/db/$CA.db.attr
  echo 01 > ca/$CA/db/$CA.srl    # numero de serie initial
  echo 01 > ca/$CA/db/$CA.crl.srl # numero de CRL initial
done
```

3.3.2 Fichier de configuration OpenSSL (un par AC)

Le cœur du TP réside dans les extensions. Extrait commenté de la configuration de la racine RSA (`RSA_Root_CA.conf`) :

```
[ca]
default_ca = root_ca

[root_ca]
certificate      = $dir/RSA_Root_CA.crt
private_key      = $dir/private/RSA_Root_CA.key
database         = $dir/db/RSA_Root_CA.db
serial           = $dir/db/RSA_Root_CA.srl
crlnumber        = $dir/db/RSA_Root_CA.crl.srl
default_md       = sha256          # hachage SHA-256 (ANSSI)
default_crl_days = 30              # CRL valable 1 mois
policy           = policy_loose

# --- profil de la racine elle-meme (auto-signature) ---
[root_ca_ext]
basicConstraints = critical,CA:TRUE,pathlen:2  # 2 niveaux de sous-AC
keyUsage        = critical,keyCertSign,cRLSign
subjectKeyIdentifier = hash
authorityKeyIdentifier = keyid:always

# --- profil applique aux AC subordonnees signees par la racine ---
[sub_ca_ext]
basicConstraints = critical,CA:TRUE,pathlen:0  # ne signe que des EE
keyUsage        = critical,keyCertSign,cRLSign
crlDistributionPoints = URI:http://crl.pki-apv.local/RSA_Root_CA.crl
authorityInfoAccess = caIssuers;URI:http://aia.pki-apv.local/RSA_Root_CA.crt
```

Note

La racine EC utilise le même squelette avec `pathlen:1`. Le fichier de `Sub_RSA_CA_1` contient en plus les profils d'entités finales : `server_ext` (TLS, `serverAuth`), `email_ext` (S/MIME, `emailProtection` + `dataEncipherment`) et le profil OCSP. Les quatre fichiers `.conf` complets sont fournis dans l'archive jointe.

3.3.3 Création de la racine RSA (auto-signée)

La requête est créée et la bi-clé RSA générée en une commande, puis la racine s'auto-signe :

```
# 1) genere la bi-cle RSA 4096 + la CSR
openssl req -new -config ca/RSA_Root_CA/RSA_Root_CA.conf \
  -newkey rsa:4096 -keyout ca/RSA_Root_CA/private/RSA_Root_CA.key -nodes \
  -out ca/csr/RSA_Root_CA.csr
# 2) auto-signature (15 ans, extensions racine)
openssl ca -batch -config ca/RSA_Root_CA/RSA_Root_CA.conf -selfsign \
  -in ca/csr/RSA_Root_CA.csr -extensions root_ca_ext -days 5479 \
  -out ca/RSA_Root_CA/RSA_Root_CA.crt
```

Vérification du certificat racine obtenu :

```
$ openssl x509 -in RSA_Root_CA.crt -noout -subject -issuer -dates
subject= C = FR, O = TP IMT Mines Ales, OU = PKI APV, CN = RSA Root CA APV
issuer = C = FR, O = TP IMT Mines Ales, OU = PKI APV, CN = RSA Root CA APV
notBefore=Jun 23 06:30:04 2026 GMT
notAfter =Jun 23 06:30:04 2041 GMT          # 15 ans
    X509v3 Basic Constraints: critical
        CA:TRUE, pathlen:2
    X509v3 Key Usage: critical
        Certificate Sign, CRL Sign
```

3.3.4 Création de la racine ECDSA (auto-signée)

OpenSSL ne génère pas la clé EC implicitement via `req` : on crée d'abord la clé P-384, puis la requête, puis l'auto-signature.

```
# 1) cle EC P-384 (secp384r1)
openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:secp384r1 \
  -out ca/EC_Root_CA/private/EC_Root_CA.key
# 2) CSR a partir de la cle existante
openssl req -new -config ca/EC_Root_CA/EC_Root_CA.conf \
  -key ca/EC_Root_CA/private/EC_Root_CA.key -out ca/csr/EC_Root_CA.csr
# 3) auto-signature (15 ans, pathlen:1)
openssl ca -batch -config ca/EC_Root_CA/EC_Root_CA.conf -selfsign \
  -in ca/csr/EC_Root_CA.csr -extensions root_ca_ext -days 5479 \
  -out ca/EC_Root_CA/EC_Root_CA.crt
```

```
$ openssl x509 -in EC_Root_CA.crt -noout -text | grep -E "Algorithm|NIST|pathlen|Sign"
    Signature Algorithm: ecdsa-with-SHA256
        ASN1 OID: secp384r1
        NIST CURVE: P-384
    CA:TRUE, pathlen:1
    Certificate Sign, CRL Sign
```

3.3.5 Création des autorités subordonnées

Chaque sous-AC génère sa clé et sa CSR, puis est signée par sa racine avec le profil `sub_ca_ext` (10 ans). Exemple pour la branche RSA :

```
openssl req -new -config ca/Sub_RSA_CA_1/Sub_RSA_CA_1.conf \
  -newkey rsa:3072 -keyout ca/Sub_RSA_CA_1/private/Sub_RSA_CA_1.key -nodes \
  -out ca/csr/Sub_RSA_CA_1.csr
# signature par la racine RSA
openssl ca -batch -config ca/RSA_Root_CA/RSA_Root_CA.conf \
  -extensions sub_ca_ext -days 3653 \
  -in ca/csr/Sub_RSA_CA_1.csr -out ca/Sub_RSA_CA_1/Sub_RSA_CA_1.crt
```

Validation des deux chaînes de subordination :

```
$ openssl verify -CAfile RSA_Root_CA.crt Sub_RSA_CA_1.crt
Sub_RSA_CA_1.crt: OK
$ openssl verify -CAfile EC_Root_CA.crt Sub_EC_CA_1.crt
Sub_EC_CA_1.crt: OK
```

3.4 Révocation par CRL (tâches 2 à 4)

Tâche 2 — émettre une entité finale par Sub_RSA_CA_1_APV. Le certificat généré se trouve dans le dossier 03-crl-ocsp : Sub_RSA_CA_1_APV-leaf-crl.crt.

```
cd /home/aurelien/Documents/certificats/03-crl-ocsp
openssl ca -batch \
  -config openssl-rsa-sub-ocsp.cnf \
  -extensions v3_leaf \
  -days 365 \
  -notext \
  -in Sub_RSA_CA_1_APV-leaf-crl.csr.pem \
  -out Sub_RSA_CA_1_APV-leaf-crl.crt
```

Tâche 3 — valider face à la chaîne d'autorité.

```
openssl verify \
  -CAfile /home/aurelien/Documents/certificats/02-pki-openssl/RSA_Root_CA_APV/certs/
  ↳ RSA_Root_CA_APV.crt \
  -untrusted /home/aurelien/Documents/certificats/02-pki-openssl/Sub_RSA_CA_1_APV/certs/
  ↳ Sub_RSA_CA_1_APV.crt \
  Sub_RSA_CA_1_APV-leaf-crl.crt
```

```
/home/aurelien/Documents/certificats/03-crl-ocsp/Sub_RSA_CA_1_APV-leaf-crl.crt: OK
```

Tâche 4 — révoquer puis générer et afficher la CRL.

```
openssl ca -batch -config openssl-rsa-sub-ocsp.cnf \
  -revoke Sub_RSA_CA_1_APV-leaf-crl.crt \
  -crl_reason keyCompromise
openssl ca -config openssl-rsa-sub-ocsp.cnf \
  -gencrl -crl_days 30 \
  -out ca-work/Sub_RSA_CA_1_APV/crl.pem
openssl crl -in ca-work/Sub_RSA_CA_1_APV/crl.pem -text -noout > crl-after-revoke.txt
```

```
Issuer: C=FR, O=TP Crypto Agents, OU=PKI OpenSSL, CN=Sub_RSA_CA_1_APV
Last Update: Jun 23 07:18:21 2026 GMT
Next Update: Jul 23 07:18:21 2026 GMT
Serial Number: 1000
Revocation Date: Jun 23 07:18:21 2026 GMT
X509v3 CRL Reason Code: Key Compromise
```

Le certificat apparaît bien dans la CRL, avec le motif *Key Compromise*.

3.5 Signature et chiffrement S/MIME (tâches 5 à 7)

Tâche 5 — certificat S/MIME avec les bons usages. Le certificat 04-smime/smime-user.crt est émis par Sub_RSA_CA_1_APV. Les extensions observées sont :

- Basic Constraints: CA:FALSE ;
- Key Usage: Digital Signature, Key Encipherment ;
- Extended Key Usage: E-mail Protection ;
- SAN email : smime-user@apv.local.

```
cd /home/aurelien/Documents/certificats/04-smime
openssl ca -batch \
  -config ../02-pki-openssl/Sub_RSA_CA_1_APV/openssl-rsa-sub.cnf \
  -extfile smime-openssl.cnf \
  -extensions v3_smime_signer \
```

```
-days 365 \  
-notext \  
-in smime-user.csr.pem \  
-out smime-user.crt
```

Tâches 6 et 7 — signature, chiffrement, déchiffrement et vérification. L'échange réel avec un autre groupe reste à compléter. Une démonstration locale a été réalisée avec un certificat destinataire de test : `recipient-test.crt`.

```
openssl smime -sign -binary -nodetach \  
-in message.txt \  
-signer smime-user.crt \  
-inkey smime-user.key.pem \  
-certfile ../02-pki-openssl/Sub_RSA_CA_1_APV/certs/Sub_RSA_CA_1_APV.crt \  
| openssl smime -encrypt -aes256 \  
-out signed-encrypted-message.pem \  
recipient-test.crt  
  
openssl smime -decrypt \  
-in signed-encrypted-message.pem \  
-recip recipient-test.crt \  
-inkey recipient-test.key.pem \  
-out decrypted-signed-message.pem  
  
openssl smime -verify \  
-in decrypted-signed-message.pem \  
-CAfile ../02-pki-openssl/RSA_Root_CA_APV/certs/RSA_Root_CA_APV.crt \  
-out verified-message.txt
```

Réponse

Le fichier `verified-message.txt` contient le message clair attendu. La démonstration locale signature + chiffrement + déchiffrement + vérification est donc réalisée. L'échange inter-groupe reste à compléter.

Note

Bonne pratique rappelée par le sujet : il faudrait idéalement générer deux certificats, un pour la signature et un pour le chiffrement. Le TP regroupe volontairement les deux usages dans un seul certificat pour simplifier la manipulation.

3.6 Révocation par OCSP (tâches 8 à 12)

Tâche 8 — certificat contenant URL CRL et URL OCSP. Le certificat testé se trouve dans le dossier `03-crl-ocsp` : `Sub_RSA_CA_1_APV-leaf-ocsp.crt`.

```
CRL Distribution Points:  
URI:http://crl.localhost:2560/Sub_RSA_CA_1_APV.crl  
Authority Information Access:  
OCSP - URI:http://ocsp.localhost:2560
```

Tâche 9 — répondeur OCSP pour Sub_RSA_CA_1_APV. Le répondeur local est lancé sur `127.0.0.1:2560`.

```
openssl ocsp \  
-index ca-work/Sub_RSA_CA_1_APV/db/index.txt \  
-port 2560 \  
-rsigner ../02-pki-openssl/Sub_RSA_CA_1_APV/certs/Sub_RSA_CA_1_APV.crt \  
-rkey ../02-pki-openssl/Sub_RSA_CA_1_APV/private/Sub_RSA_CA_1_APV.key.pem \  
-CA ../02-pki-openssl/Sub_RSA_CA_1_APV/certs/Sub_RSA_CA_1_APV.crt \  
-nrequest 1
```

Tâche 10 — vérifier : statut attendu *good*.

```
openssl ocsp \
  -issuer ../02-pki-openssl/Sub_RSA_CA_1_APV/certs/Sub_RSA_CA_1_APV.crt \
  -cert Sub_RSA_CA_1_APV-leaf-ocsp.crt \
  -url http://127.0.0.1:2560 \
  -CAfile ../02-pki-openssl/RSA_Root_CA_APV/certs/RSA_Root_CA_APV.crt \
  -verify_other ../02-pki-openssl/Sub_RSA_CA_1_APV/certs/Sub_RSA_CA_1_APV.crt \
  -resp_text -text -no_nonce
```

```
Cert Status: good
Response verify OK
Sub_RSA_CA_1_APV-leaf-ocsp.crt: good
```

Tâches 11 et 12 — révoquer puis re-vérifier : statut attendu *revoked*.

```
openssl ca -batch -config openssl-rsa-sub-ocsp.cnf \
  -revoke Sub_RSA_CA_1_APV-leaf-ocsp.crt \
  -crl_reason cessationOfOperation
```

```
Cert Status: revoked
Revocation Time: Jun 23 07:18:22 2026 GMT
Response verify OK
Sub_RSA_CA_1_APV-leaf-ocsp.crt: revoked
```

Le statut passe bien de *good* à *revoked*.

3.7 Certification croisée (tâches 13 et 14)

Tâche 13 — entité finale émise par Sub_EC_CA_1_APV. Le certificat final émis pour tester les chemins est 05-cross-cert/certs/Leaf_EC_1_APV.crosspath.crt.

Mise en place du pont croisé. La racine RSA_Root_CA_APV signe un certificat croisé pour EC_Root_CA_APV. On obtient ainsi un second certificat pour la racine EC, mais émis par la racine RSA :

```
05-cross-cert/certs/EC_Root_CA_APV.cross-signed-by-RSA_Root_CA_APV.crt
```

Tâche 14 — vérifier les deux chaînes possibles.

```
# Chaîne native : Leaf_EC -> Sub_EC_CA_1 -> EC_Root_CA
openssl verify -show_chain \
  -CAfile /home/aurelien/Documents/certificats/02-pki-openssl/EC_Root_CA_APV/certs/EC_Root_CA_APV
  ↪ .crt \
  -untrusted /home/aurelien/Documents/certificats/02-pki-openssl/Sub_EC_CA_1_APV/certs/
  ↪ Sub_EC_CA_1_APV.crt \
  /home/aurelien/Documents/certificats/05-cross-cert/certs/Leaf_EC_1_APV.crosspath.crt

# Chaîne croisee : Leaf_EC -> Sub_EC_CA_1 -> EC_Root_CA cross-signe -> RSA_Root_CA
openssl verify -show_chain \
  -CAfile /home/aurelien/Documents/certificats/02-pki-openssl/RSA_Root_CA_APV/certs/
  ↪ RSA_Root_CA_APV.crt \
  -untrusted /home/aurelien/Documents/certificats/05-cross-cert/certs/untrusted-rsa-path.pem \
  /home/aurelien/Documents/certificats/05-cross-cert/certs/Leaf_EC_1_APV.crosspath.crt
```

```
/home/aurelien/Documents/certificats/05-cross-cert/certs/Leaf_EC_1_APV.crosspath.crt: OK
depth=0: CN=Leaf_EC_1_APV
depth=1: CN=Sub_EC_CA_1_APV
depth=2: CN=EC_Root_CA_APV

/home/aurelien/Documents/certificats/05-cross-cert/certs/Leaf_EC_1_APV.crosspath.crt: OK
depth=0: CN=Leaf_EC_1_APV
depth=1: CN=Sub_EC_CA_1_APV
```

```
depth=2: CN=EC_Root_CA_APV
depth=3: CN=RSA_Root_CA_APV
```

Note

La certification croisée réalisée ici est **unidirectionnelle**. Pour une croisée bidirectionnelle, il faudrait émettre le certificat miroir `RSA_Root_CA_APV.cross-signed-by-EC_Root_CA_APV.crt`.

3.8 Magasin de certificats Windows (tâche 15)

Cette partie n'a pas été exécutée sur Windows dans l'environnement Linux. Une procédure a été préparée pour la VM Windows Server.

► À COMPLÉTER (non réalisable hors ligne / hors VM)

Certificats à exporter/importer :

- racine : `RSA_Root_CA_APV.crt` ou `HSM_Root_CA_APV.crt` selon la chaîne testée ;
- intermédiaire : `Sub_RSA_CA_1_APV.crt` ou l'AC ADCS subordonnée une fois émise.

Magasins Windows visés :

- racine : Autorités de certification racines de confiance / Cert:
LocalMachine
Root ;
- intermédiaire : Autorités de certification intermédiaires / Cert:
LocalMachine
CA.

```
Import-Certificate -FilePath C:\PKI\HSM_Root_CA_APV.cer -CertStoreLocation Cert:\
↳ LocalMachine\Root
Import-Certificate -FilePath C:\PKI\HSM_Sub_CA_APV.cer -CertStoreLocation Cert:\
↳ LocalMachine\CA
```

Validation attendue : ouverture du certificat dans Windows et vérification de l'onglet **Chemin d'accès de certification**. Captures à compléter : magasin racine, magasin intermédiaire et chaîne validée.

4 Partie 3 — SoftHSM2 et ADCS

4.1 Autorité racine Linux avec SoftHSM2

Le travail réalisé sous Linux couvre : configuration locale de SoftHSM2, initialisation d'un token dédié, génération d'une clé RSA 4096 bits dans le token, émission d'un certificat racine autosigné, génération d'une CRL/ARL valable environ un mois, utilisation d'OpenSSL via PKCS#11 et vérification de l'absence de fichier de clé privée exporté hors SoftHSM2.

```
cd /home/aurelien/Documents/certificats/06-softhsm
./scripts/bootstrap-hsm-root-ca.sh
```

```
Subject: C=FR, O=TP Crypto, OU=06-softhsm, CN=HSM_Root_CA_APV
Issuer:  C=FR, O=TP Crypto, OU=06-softhsm, CN=HSM_Root_CA_APV
Cle: RSA 4096 bits
Validite: Jun 23 07:17:04 2026 GMT -> Jun 20 07:17:04 2036 GMT
Extensions: CA:TRUE, pathlen:1, Certificate Sign, CRL Sign
```

4.2 CRL / ARL de la racine HSM

```
Issuer: C=FR, O=TP Crypto, OU=06-softhsm, CN=HSM_Root_CA_APV
Last Update: Jun 23 07:17:04 2026 GMT
Next Update: Jul 23 07:17:04 2026 GMT
No Revoked Certificates.
```

4.3 Absence d'export de clé privée

```
sh scripts/verify-no-private-files.sh
```

```
No private key file (.key or private .pem) found outside SoftHSM2.
```

Note

Aucun PIN, mot de passe, secret de token ou contenu de clé privée n'est inclus dans ce compte rendu. Les commandes qui nécessitent un PIN doivent utiliser une variable d'environnement locale ou une saisie interactive, jamais une valeur en clair dans le rapport.

4.4 Point technique PKCS#11

La tentative avec le provider `pkcs11` a échoué car le module attendu n'existait pas sous ce nom. La solution utilisée est le provider OpenSSL 3 `pkcs11prov`, avec le module SoftHSM2 configuré.

4.5 Autorité subordonnée ADCS Windows

Cette partie est à réaliser sur Windows Server 2019. Le dossier `07-windows-handoff/README.md` contient une procédure de préparation, mais pas de preuve d'exécution.

► À COMPLÉTER (non réalisable hors ligne / hors VM)**Procédure attendue :**

1. Installer ADCS en mode *Standalone CA* :

```
Install-WindowsFeature ADCS-Cert-Authority -IncludeManagementTools
```

2. Créer une AC subordonnée autonome avec clé RSA 4096 bits, SHA-256 et validité 5 ans :

```
Install-AdcsCertificationAuthority '  
-CAType StandaloneSubordinateCA '  
-CACommonName "HSM_Sub_CA_APV" '  
-KeyLength 4096 '  
-HashAlgorithmName SHA256 '  
-CryptoProviderName "RSA#Microsoft Software Key Storage Provider" '  
-OutputCertRequestFile C:\PKI\HSM_Sub_CA_APV.req '  
-ValidityPeriod Years '  
-ValidityPeriodUnits 5
```

3. Transférer la CSR PKCS#10 vers Linux et la signer avec la racine OpenSSL/SoftHSM2 :

```
cd /home/aurelien/Documents/certificats/06-softhsm  
./scripts/sign-adcs-csr.sh HSM_Sub_CA_APV.req issued/HSM_Sub_CA_APV.crt
```

4. Renvoyer le certificat signé sur Windows et finaliser ADCS :

```
certreq -accept C:\PKI\HSM_Sub_CA_APV.cer
```

5. Générer une requête TLS avec **certreq**, puis la soumettre à l'AC ADCS :

```
certreq -new C:\PKI\tls.inf C:\PKI\tls.req  
certutil -config "NomServeur\HSM_Sub_CA_APV" -submit C:\PKI\tls.req C:\PKI\tls.cer
```

Captures nécessaires : rôle ADCS installé, AC subordonnée configurée, certificat racine importé, certificat subordonné valide, certificat TLS émis et chaîne visible dans Windows.

5 Conclusion

Sous Linux, le TP a permis d'analyser des certificats publics, de vérifier CRL et OCSP, de construire une PKI OpenSSL, de révoquer des certificats, de réaliser une démonstration S/MIME locale, de tester une certification croisée et de créer une racine protégée par SoftHSM2.

Réalisé et vérifié :

- analyse du certificat `cyber.gouv.fr`, de sa chaîne GandiCert / DigiCert, de sa CRL et de son statut de révocation ;
- vérification CRL et OCSP d'un certificat révoqué `SSL.com` ;
- comparaison des certificats `google.com` et `youtube.com` ;
- création des AC RSA et EC OpenSSL, puis vérification des chaînes ;
- révocation CRL locale et répondeur OCSP local avec passage de `good` à `revoked` ;
- démonstration S/MIME locale ;
- certification croisée unidirectionnelle et vérification des deux chemins pour le certificat EC final ;
- racine Linux SoftHSM2 et CRL/ARL valable environ un mois.

À compléter :

- captures navigateur de `revoked-rsa-dv.ssl.com` ;
- capture navigateur de `wrong.host.badssl.com` ;
- analyse de `isrg-1.crt` et `isrg-2.crt` si les fichiers sont fournis ;
- échange S/MIME réel avec un autre groupe ;
- import et validation dans le magasin Windows ;
- AC subordonnée ADCS Windows Server 2019 et certificat TLS émis par ADCS.

A Annexe — Arborescence des fichiers produits

```
/home/aurelien/Documents/certificats/
|-- 01-public-certs/
|   |-- cyber.gouv.fr.pem, cyber.gouv.fr.chain.txt, cyber.gouv.fr.x509.txt
|   |-- GandiCert.crl, crl.txt
|   |-- revoked-rsa-dv.ssl.com.pem, revoked-rsa-dv.ssl.com.crl.txt, revoked-rsa-dv.ssl.com.ocsp.txt
|   |-- google.com.pem, youtube.com.pem, wrong.host.badssl.com.openssl.txt
|-- 02-pki-openssl/
|   |-- RSA_Root_CA_APV/      (certs/RSA_Root_CA_APV.crt, db/, private/, openssl-rsa-root.cnf)
|   |-- Sub_RSA_CA_1_APV/    (certs/Sub_RSA_CA_1_APV.crt, certs/Leaf_RSA_1_APV.crt)
|   |-- EC_Root_CA_APV/     (certs/EC_Root_CA_APV.crt, openssl-ec-root.cnf)
|   |-- Sub_EC_CA_1_APV/    (certs/Sub_EC_CA_1_APV.crt, certs/Leaf_EC_1_APV.crt)
|-- 03-crl-ocsp/
|   |-- Sub_RSA_CA_1_APV-leaf-crl.crt, Sub_RSA_CA_1_APV-leaf-ocsp.crt
|   |-- verify-before-revoke.txt, crl-after-revoke.txt, ocsp-good.txt, ocsp-revoked.txt
|-- 04-smime/
|   |-- smime-user.csr.pem, smime-user.crt, recipient-test.crt
|   |-- signed-encrypted-message.pem, decrypted-signed-message.pem, verified-message.txt
|-- 05-cross-cert/
|   |-- certs/EC_Root_CA_APV.cross-signed-by-RSA_Root_CA_APV.crt
|   |-- certs/Leaf_EC_1_APV.crosspath.crt, certs/untrusted-rsa-path.pem
|   |-- verify/verify-ec-self.txt, verify/verify-rsa-cross.txt
|-- 06-softhsm/
```

```
| |-- softhsm2.conf, openssl-hsm-root.cnf  
| |-- state/certs/HSM_Root_CA_APV.crt, state/crl/HSM_Root_CA_APV.crl  
| |-- scripts/bootstrap-hsm-root-ca.sh, sign-adcs-csr.sh, verify-no-private-files.sh  
|-- 07-windows-handoff/  
| |-- README.md
```

B Annexe — Captures restantes à insérer

- Navigateurs sur `revoked-rsa-dv.ssl.com` : Firefox, Chromium, troisième navigateur.
- Navigateur sur `wrong.host.badssl.com`.
- Magasin de certificats Windows.
- Installation et configuration ADCS.
- Certificat TLS Windows.
- Échange S/MIME réel avec un autre groupe.

Note

Le trigramme utilisé dans ce compte rendu est **APV**. Les chemins de fichiers ont été adaptés à la machine : `/home/aurelien/Documents/certificats`.